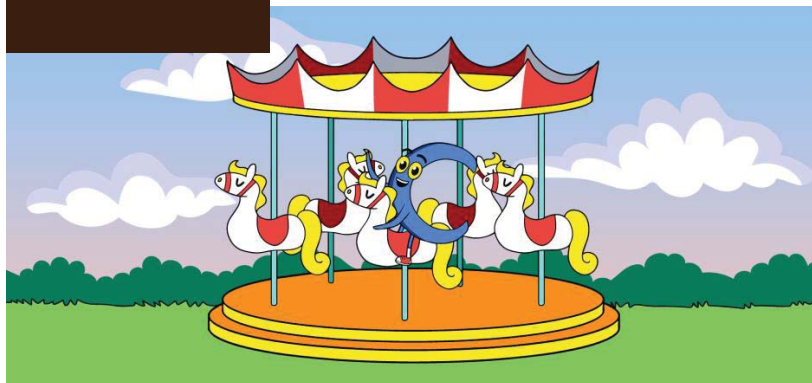


5

Loop Control Instruction



"Merry go round..."

If you wish to find averages of 100 sets of three numbers, would you actually execute the program 100 times? Obviously not, there has to be a better way out. This chapter will show you how to accomplish this...

**Contents**

- Loops
- The *while* Loop
 - Tips and Traps
 - More Operators
- Programs
- Exercises
- KanNotes

The programs that we have developed so far used either a sequential or a decision control instruction. These programs performed the same series of actions, in the same way, exactly once. In programming, we frequently need to perform an action over and over, often with variations in the details each time. The mechanism which meets this need is the 'Loop Control Instruction', and loops are the subject of this chapter.

Loops

The versatility of computer lies in its ability to perform a set of instructions repeatedly. This involves repeating some portion of the program either a specified number of times or until a particular condition is satisfied. This repetitive operation is done through a loop control instruction. There are three ways to repeat a part of a program. They are:

- (a) Using a **while** statement
- (b) Using a **for** statement
- (c) Using a **do-while** statement

Let us begin by understanding the **while** loop.

The *while* Loop

The **while** loop is ideally suited for situations where we wish to repeat some instructions till a condition remains true. Let us understand this with the help of an example. Suppose we wish to calculate simple interest for 3 sets of values of principal, number of years and rate of interest. Here is the program that can achieve this...

```
/* Calculation of simple interest for 3 sets of p, n and r */
#include <stdio.h>
int main( )
{
    int  p, n, count ;
    float  r, si ;
    count = 1 ;
    while ( count <= 3 )
    {
        printf ( "\nEnter values of p, n and r " ) ;
        scanf ( "%d %d %f", &p, &n, &r ) ;
        si = p * n * r / 100 ;
        printf ( "Simple interest = Rs. %\nf", si ) ;
```

```
        count = count + 1 ;  
    }  
    return 0 ;  
}
```

Given below is the interaction with this program on its execution.

```
Enter values of p, n and r 1000 5 13.5  
Simple interest = Rs. 675.000000  
Enter values of p, n and r 2000 5 13.5  
Simple interest = Rs. 1350.000000  
Enter values of p, n and r 3500 5 3.5  
Simple interest = Rs. 612.500000
```

The program executes all statements after the **while** 3 times. The logic for calculating the simple interest is written in these statements and they are enclosed within { }. These statements form the 'body' of the **while** loop. The parentheses after the **while** contain a condition. So long as this condition remains true, the statements in the body of the **while** loop keep getting executed repeatedly. To begin with, the variable **count** is initialized to 1 and every time the simple interest logic is executed, the value of **count** is incremented by one. The variable **count** is often called either a 'loop counter' or an 'index variable'.

The operation of the **while** loop is illustrated in Figure 5.1.

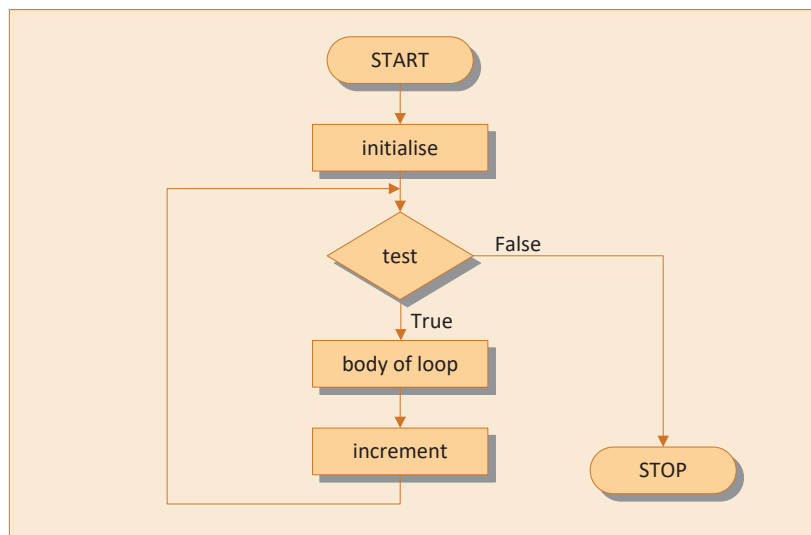


Figure 5.1 Operation of *while* loop.

Tips and Traps

Note the following points about **while...**

- The statements within the **while** loop would keep getting executed till the condition being tested remains true. When the condition becomes false, the control passes to the first statement that follows the body of the **while** loop.
- Almost always, the **while** must test a condition that will eventually become false, otherwise the loop would keep getting executed forever, indefinitely.

```
int i = 1 ;
while ( i <= 10 )
    printf ( "%d\n", i ) ;
```

This is an indefinite loop, since **i** always remains equal to 1. The correct form would increment **i** in the body of the loop.

- Instead of incrementing a loop counter, we can decrement it and still manage to get the body of the loop executed repeatedly. This is shown below.

```
int i = 5 ;
while ( i >= 1 )
{
    printf ( "Make the computer literate!\n" ) ;
    i = i - 1 ;
}
```

- It is not necessary that a loop counter must only be an **int**. It can even be a **float**.

```
float a = 10.0 ;
while ( a <= 10.5 )
{
    printf ( "Raindrops on roses..." ) ;
    printf ( "...and whiskers on kittens\n" ) ;
    a = a + 0.1 ;
}
```

- Even floating-point loop counters can be decremented. Once again, the increment and decrement could be by any suitable value, not necessarily 1.
- Multiple conditions can be used in the **while** loop by connecting them using logical operators as shown in the following examples:

```
while ( i >= 10 && j <= 15 )
while ( j > 10 && ( b < 15 || c < 20 ) )
```

- If there is only one statement within the loop then { } are optional.
- What will be the output of the following program?

```
# include <stdio.h>
int main( )
{
    int i = 1 ;
    while ( i <= 10 ) ;
    {
        printf ( "%d\n", i ) ;
        i = i + 1 ;
    }
    return 0 ;
}
```

This is an indefinite loop, and it doesn't give any output at all. The reason is, we have carelessly put a ; after the **while**. It would make the loop work like this...

```
while ( i <= 10 )
;
{
    printf ( "%d\n", i ) ;
    i = i + 1 ;
}
```

Since the value of **i** is not getting incremented, the control would keep rotating within the loop eternally. Note that enclosing **printf()** and **i = i + 1** within a pair of braces is not an error. In fact, we can put a pair of braces around any individual statement or set of statements without affecting the execution of the program.

More Operators

There are several operators that are frequently used with **while**. To illustrate their use, let us consider a problem wherein numbers from 1 to 10 are to be printed on the screen. The program for performing this task can be written using **while** in following different ways:

```
(a) #include <stdio.h>
int main( )
{
    int i = 1 ;
    while ( i <= 10 )
    {
        printf ( "%d\n", i );
        i = i + 1 ;
    }
    return 0 ;
}
```

This is the most straight-forward way of printing numbers from 1 to 10.

```
(b) #include <stdio.h>
int main( )
{
    int i = 1 ;
    while ( i <= 10 )
    {
        printf ( "%d\n", i );
        i++ ;
    }
    return 0 ;
}
```

Note that every time the statement **i++** gets executed the increment operator **++** increments the value of **i** by 1. Similarly, to reduce the value of a variable by 1, a decrement operator **--** is also available.

However, never use **n+++** to increment the value of **n** by 2, since there doesn't exist an operator **+++** in C.

```
(c) #include <stdio.h>
```

```
int main( )
{
    int i = 1 ;
    while ( i <= 10 )
    {
        printf ( "%d\n", i ) ;
        i += 1 ;
    }
    return 0 ;
}
```

Note that `+=` is a compound assignment operator. It increments the value of `i` by 1. Similarly, `j = j + 10` can also be written as `j += 10`. Other compound assignment operators are `-=`, `*=`, `/=` and `%=`.

(d) #include <stdio.h>

```
int main( )
{
    int i = 0 ;
    while ( i++ < 10 )
        printf ( "%d\n", i ) ;
    return 0 ;
}
```

In the statement **while (i++ < 10)**, first the comparison of value of `i` with 10 is performed, and then the incrementation of `i` takes place. Since the incrementation of `i` happens after the comparison, here the `++` operator is called a post-incrementation operator. When the control reaches **printf()**, `i` has already been incremented, hence `i` must be initialized to 0, not 1.

(e) #include <stdio.h>

```
int main( )
{
    int i = 0 ;
    while ( ++i <= 10 )
        printf ( "%d\n", i ) ;
    return 0 ;
}
```

In the statement **while (++i <= 10)**, first incrementation of `i` takes place, then the comparison of value of `i` with 10 is performed. Since

the incrementation of **i** happens before the comparison, here the **++** operator is called a pre-incrementation operator.

P Programs

Problem 5.1

Write a program to calculate overtime pay of 10 employees. Overtime is paid at the rate of Rs. 120.00 per hour for every hour worked above 40 hours. Assume that employees do not work for fractional part of an hour.

Program

```
/* Determine overtime pay of 10 employees */
#include <stdio.h>
int main( )
{
    float otpay ;
    int hour, i = 1 ;
    while ( i <= 10 ) /* Loop for 10 employees */
    {
        printf ( "\nEnter no. of hours worked: " ) ;
        scanf ( "%d", &hour ) ;
        if ( hour >= 40 )
            otpay = ( hour - 40 ) * 120 ;
        else
            otpay = 0 ;
        printf ( "Hours = %d Overtime pay = Rs.%f\n", hour, otpay ) ;
        i++ ;
    }
    return 0 ;
}
```

Output

```
Enter no. of hours worked: 45
Hours = 45 Overtime pay = Rs.600.000000

Enter no. of hours worked: 50
Hours = 50 Overtime pay = Rs.1200.000000
```

```
Enter no. of hours worked: 20
Hours = 20 Overtime pay = Rs.0.000000
```

Problem 5.2

Write a program to find the factorial value of any number entered through the keyboard.

Program

```
/* Calculation of factorial value of a number */
#include <stdio.h>
int main( )
{
    int num, i, fact ;
    printf ( "Enter a number: " ) ;
    scanf ( "%d", &num ) ;
    fact = i = 1 ;
    while ( i <= num )
    {
        fact = fact * i ;
        i++ ;
    }
    printf ( "Factorial value of %d = %d\n", num, fact ) ;
    return 0 ;
}
```

Output

```
Enter a number: 7
Factorial value of 7 = 5040
```

Problem 5.3

Two numbers are entered through the keyboard. Write a program to find the value of one number raised to the power of another.

Program

```
/* Compute value of one number raised to another */
```

```
#include <stdio.h>
int main( )
{
    float x, power ;
    int y, i ;
    printf ( "\nEnter two numbers: " ) ;
    scanf ( "%f %d", &x, &y ) ;
    power = i = 1 ;
    while ( i <= y )
    {
        power = power * x ;
        i++ ;
    }
    printf ( "%f to the power %d is %f\n", x, y, power ) ;
    return 0 ;
}
```

Output

```
Enter two numbers: 2.5 3
2.500000 to the power 3 is 15.625000
```

Exercises

[A] What will be the output of the following programs?

(a) #include <stdio.h>
int main()
{
 int i = 1 ;
 while (i <= 10) ;
 {
 printf ("%d\n", i) ;
 i++ ;
 }
 return 0 ;
}

(b) #include <stdio.h>
int main()

```
{
    int x = 4, y, z ;
    y = --x ;
    z = x-- ;
    printf ( "%d %d %d\n", x, y, z ) ;
    return 0 ;
}
```

(c) # include <stdio.h>
int main()
{
 int x = 4, y = 3, z ;
 z = x-- - y ;
 printf ("%d %d %d\n", x, y, z) ;
 return 0 ;
}

(d) # include <stdio.h>
int main()
{
 while ('a' < 'b')
 printf ("malayalam is a palindrome\n") ;
 return 0 ;
}

(e) # include <stdio.h>
int main()
{
 int i ;
 while (i = 10)
 {
 printf ("%d\n", i) ;
 i = i + 1 ;
 }
 return 0 ;
}

(f) # include <stdio.h>
int main()
{
 float x = 1.1 ;
 while (x == 1.1)
 {

```
        printf ( "%f\n", x );
        x = x - 0.1 ;
    }
    return 0 ;
}
```

[B] Attempt the following questions:

- (a) Write a program to print all the ASCII values and their equivalent characters using a while loop. The ASCII values vary from 0 to 255.
- (b) Write a program to print out all Armstrong numbers between 1 and 500. If sum of cubes of each digit of the number is equal to the number itself, then the number is called an Armstrong number. For example, $153 = (1 * 1 * 1) + (5 * 5 * 5) + (3 * 3 * 3)$.
- (c) Write a program for a matchstick game being played between the computer and a user. Your program should ensure that the computer always wins. Rules for the game are as follows:
 - There are 21 matchsticks.
 - The computer asks the player to pick 1, 2, 3, or 4 matchsticks.
 - After the person picks, the computer does its picking.
 - Whoever is forced to pick up the last matchstick loses the game.
- (d) Write a program to enter numbers till the user wants. At the end it should display the count of positive, negative and zeros entered.
- (e) Write a program to receive an integer and find its octal equivalent.
(Hint: To obtain octal equivalent of an integer, divide it continuously by 8 till dividend doesn't become zero, then write the remainders obtained in reverse direction.)
- (f) Write a program to find the range of a set of numbers entered through the keyboard. Range is the difference between the smallest and biggest number in the list.



- Repetition control instruction is used to repeat a set of statements in a program.
- It is implemented using
 - i) while loop

- 2) for loop
- 3) do - while loop

- General form of while:

```
i = 1; /* initialization of loop counter */
while (i <= 10) /* testing of loop counter */
{
    statement1;
    statement2;
    i++; /* incrementation of loop counter */
}
```

- i++ increments value by 1
- i-- decrements value of i by 1
- There are no **, // and %% operators
- The expressions i = i + 1, i++ and ++i are all same
- j = ++i; first increments i, then assigns the incremented value to j
- j = i++; first assigns current value of i to j, then increments i
- while (++i < 10) first increments i, then checks condition
- while (i++ < 10) first checks condition, then increments i
- i = i + 5 is same as i += 5
- Compound assignment operators: +=, -=, *=, /= and % =
- Running sum and products are implemented using following :


```
s = 0;
p = 1;
while (condition)
{
    /* calculate term */
    s = s + term;
    p = p * term;
}
```